

1. Project Description

The **Bloom Shop** is a full-stack e-commerce web application specifically designed for a flower boutique. The project provides a seamless shopping experience where users can browse floral arrangements by categories (such as roses, wedding buckets, and bestsellers), view detailed product specifications, manage a shopping cart, and simulate a secure payment process.

The system is built using a modern decoupled architecture:

Backend: Developed with **Spring Boot** and **Java**, utilizing **PostgreSQL** for persistent data storage. It provides a RESTful API to manage product data and handle category-based filtering.

Frontend: Built with **React** and **Material UI (MUI)**. It features a responsive design, dynamic routing with React Router, and state management via React Context API for the shopping cart.

2. Important & Unique Features

What sets this project apart from a standard e-commerce template are the following custom implementations:

Custom Database Schema: While based on a provided template, the database was extended with a `details` column (TEXT type) to support long-form product descriptions and a `featured` boolean to control homepage highlights.

Dynamic Category Filtering: The system implements a dynamic routing mechanism where categories like "Wedding Buckets" are fetched from the backend using URL parameters, ensuring the frontend is scalable as new categories are added to the database.

Advanced UI/UX: Utilizing Material UI, the project features a "Glassmorphism" inspired sidebar, a sticky header with a real-time badge count for the basket, and a custom "Playfair Display" serif typography to match the boutique aesthetic.

Robust State Management: The `CartContext` manages global state, allowing users to add, remove, increment, or decrement items from any page without losing data during navigation.

Data Integrity: The backend includes custom Exception Handling (e.g., `ResponseStatusException`) to ensure that if a product is not found, the frontend receives a proper 404 error rather than a generic server crash.

Backend Framework: Spring Boot (Java 17+)
Database: PostgreSQL
Frontend Library: React.js
UI Framework: Material UI (MUI)
Data Persistence: Spring Data JPA / Hibernate
API Architecture: RESTful (JSON)
Routing: React Router DOM v6
State Management: React Context API (for Shopping Cart)
CORS Configuration: Enabled for localhost:3000 & 3001
Key Models: Product (id, name, price, description, details, category, featured)
Main API Endpoints:
/api/products, /api/products/top, /api/products/category/{name}

4. System Architecture Overview

The application follows a standard Three-Tier Architecture. The **Presentation Layer** (React) communicates with the **Application Layer** (Spring Boot Controllers) via asynchronous Fetch API calls. The **Data Access Layer** (JPA Repositories) interacts with the **PostgreSQL Database** to perform CRUD operations and filtered queries.